



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Building a Big Data Architecture Second part

BDM Project Document

Adrián Segura, Pau Adal, Pablo Rodríguez

08-06-2026

Master in Data Science

Contents

1	Architecture design (already improved as you told us to do it)	1
1.1	Landing Zone	1
1.2	Trusted Zone	1
1.3	Exploitation Zone	2
1.4	Consumption Layer	3
2	Trusted Zone	4
2.1	Structured Data Processing	4
2.2	Semi-Structured Data Processing	5
2.3	Unstructured Data Processing	7
3	Exploitation Zone	9
3.1	Structured and Semi-Structured Integration	9
3.2	Semi-Structured Data Enrichment	10
3.3	Unstructured Data Extraction	10
4	Data Consumption	12
4.1	Predictive Analysis: Collision Risk Modeling	12
4.2	Analytical Dashboard: Traffic Collision Insights	13
4.3	NLP and STT	13
4.4	Real-Time Analysis and Alerts	14
5	Data Governance Strategy	15

Instructions

The complete source code, configuration files, and data orchestration logic for this project are hosted in the following GitHub repository:

<https://github.com/PabloRodriguezElvira/BDM-LandingZoneProject>

Execution Guide: All the necessary instructions to deploy the infrastructure (Docker, Kafka, Airflow, and MinIO), configure the environment, and run the streaming pipeline are detailed in the `README.md` file located in the root of the repository and inside of the zip delivered.

1 Architecture design (already improved as you told us to do it)

In this section, we present the updated architecture of our project (see Figure 1). For this phase, we decided to follow the "Moving the Data Out" strategy. Instead of keeping everything inside a single Data Lake, we move the data to different specialized databases. This helps us get much better performance when querying the data later. The pipeline is divided into several zones, from the raw data ingestion to the final applications.

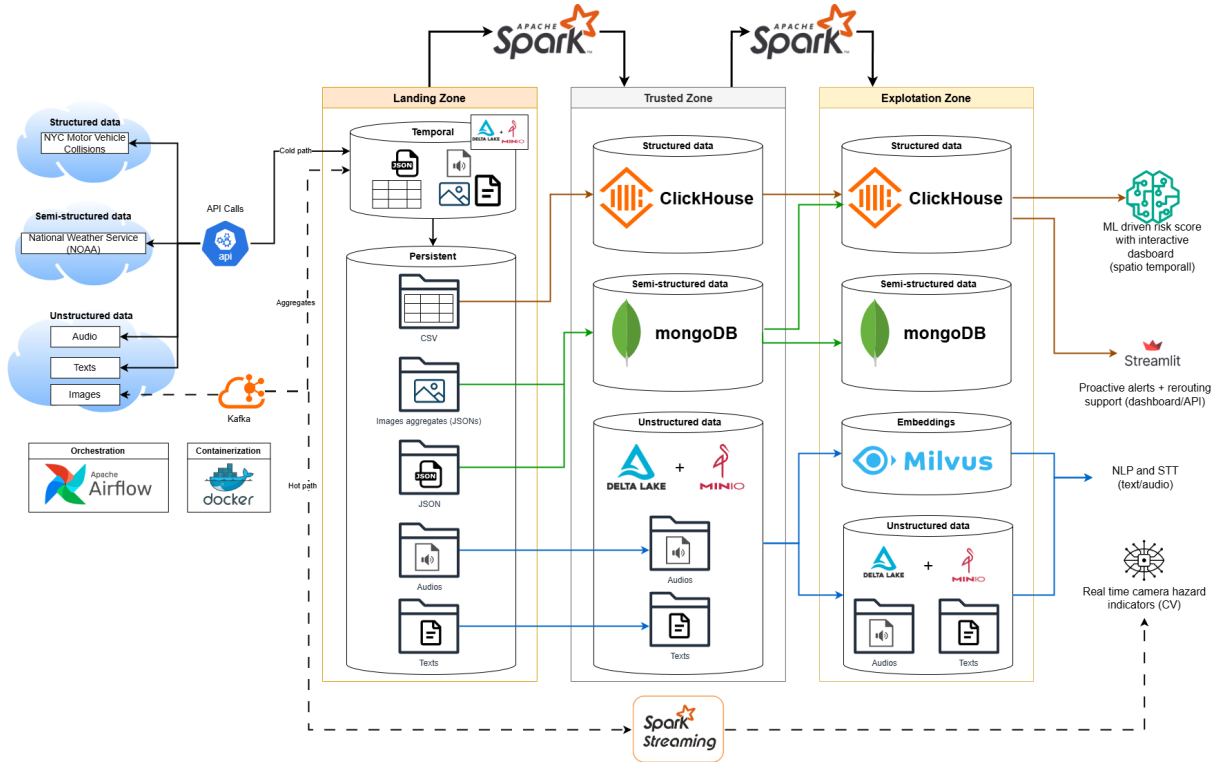


Figure 1: End-to-end Big Data Architecture for NYC Urban Monitoring.

1.1 Landing Zone

As explained in the first deliverable, the Landing zone is our entry point for all raw data. We use a Cold Path for batch ingestion, storing the 3 types of data directly in MinIO. This ensures we keep a permanent copy of the original data for future needs. For real-time camera data, we use a Hot-Warm Path powered by Apache Kafka where real-time images will be used to detect real-time alerts. Also, it is important to highlight, that from this real-time images aggregated metadata has been extracted in JSON format stored in the persistent Landing zone.

1.2 Trusted Zone

The Trusted Zone is our central layer for cleaning, standardizing, and ensuring data quality. We use Apache Spark to process raw data from the Landing Zone, applying rigorous rules to handle errors and missing values. A key principle here is data lineage: any record that fails validation (e.g., corrupted files, unparseable strings, or invalid formats)

is isolated in a "rejected" or "skipped" folder in MinIO for auditing, ensuring that no data is lost while maintaining a high-quality pipeline.

- **ClickHouse (Structured Data):** We process NYC collision records by removing duplicates, casting types (e.g., latitude/longitude to Double, metrics to Integer), and normalizing dates to YYYY-MM-DD. This standardization ensures that geographic and temporal fields are perfectly aligned, enabling seamless joins in the next layer.
- **MongoDB (Semi-structured Data):** We process JSON-based environmental logs and camera telemetry. For weather data, we use Regular Expressions to extract numerical values from strings (e.g., converting "15 mph" to a numeric 15) and flatten nested structures. For camera metadata, we perform ID normalization—mapping complex camera IDs to specific NYC Boroughs and assigning a standard "UNKNOWN" label for unrecognized IDs to prevent pipeline crashes.
- **MinIO (Unstructured Data):** We validate physical assets to ensure they meet minimum quality standards. For 911 audio, we enforce a strict baseline by converting files to mono, 16-bit PCM, and validating minimum duration. For text, we perform encoding validation (UTF-8) and whitespace normalization. Files failing these checks are isolated with a skip reason for future auditing, guaranteeing that only high-quality assets proceed to vectorization.

By standardizing these datasets early and isolating errors, we create a robust foundation. This allows the Exploitation Zone to join and analyze this information efficiently, preventing potential system crashes caused by poor data quality.

1.3 Exploitation Zone

Once cleaned in the Trusted Zone, data is moved to the Exploitation Zone to be transformed into high-value business assets. We will use Apache spark as the primary engine to model, aggregate, and enrich our datasets, ensuring they are optimized for complex analytical queries and ready for the Consumption Layer.

- **Structured and Semi-Structured Integration:** Our goal is to create query-friendly analytical tables. Using Spark, we perform distributed joins between structured collision records (ClickHouse) and semi-structured metadata (MongoDB). By aligning these via a geographic key, we generate a "Master Table" that enriches historical accident records with environmental and traffic context. Furthermore, we compute pre-calculated KPIs, such as total traffic volume, to provide analysts with immediate, high-performance insights.
- **Semi-Structured Data Enrichment:** We maintain a flexible NoSQL approach. We use Spark to calculate aggregated traffic metrics, which are then persisted in MongoDB. This ensures that the traffic data remains structurally consistent and ready for independent querying, allowing our dashboards to retrieve the latest city sensor states without performing heavy computations on the fly.
- **Unstructured Data Extraction:** To bridge the gap between raw files and ML-ready assets, we transform text and audio into mathematical representations. Using Spark, we apply custom vectorization—a hashing-based approach for text and

acoustic statistics for audio—to convert these files into numerical embeddings. These vectors are stored in **Milvus** for fast similarity searching, while original assets remain in **MinIO** for visual retrieval, ensuring a complete pipeline from raw media to searchable intelligence.

1.4 Consumption Layer

The Consumption Layer is the final stage where we transform prepared data into actionable insights. Its primary goal is to prove our pipeline’s end-to-end functionality by powering interactive tools and automated decision-making workflows directly from the Exploitation Zone assets.

- **Predictive Risk Scoring:** We implemented a machine learning module that connects to ClickHouse to train binary classification models (Random Forest and XGBoost). It predicts collision risk based on environmental factors (e.g., precipitation, wind) and temporal variables. An interactive simulator in our dashboard allows users to adjust real-world parameters to visualize how risk levels shift across different boroughs.
- **Analytical Dashboard:** Using Streamlit and Plotly, we built a business intelligence tool that queries our "Master Risk Table." It provides monthly trend analyses and identifies the top contributing factors to accidents per borough. By using caching mechanisms (`st.cache_data`), the dashboard ensures high performance even when visualizing historical trends.
- **NLP and STT Analysis:** Leveraging the vector embeddings stored in Milvus, we perform semantic incident analysis. For news articles, we apply NLP techniques to cluster reports by context, allowing managers to identify recurring themes. For 911 audio recordings, we integrate STT models to transcribe emergency calls, enabling us to filter reports by specific keywords like "medical emergency" or "collision."
- **Real-Time Traffic Alerts:** Through Apache Spark Streaming and Kafka, we analyze live camera feeds using the MobileNet-SSD model. The system detects vehicle density in 15-second windows; when a threshold of 20 vehicles is exceeded, it triggers an automated alert, saving both a diagnostic report and the associated evidence image into a structured directory for immediate response.

2 Trusted Zone

After storing the raw files in the Landing Zone (persistent), the data pipeline moves to the Trusted Zone. The main goal of this intermediate layer is to clean, standardize, and improve the quality of the data. Because raw datasets usually contain mistakes, missing values, or different formats, they must be processed carefully to fix these issues.

Following data engineering best practices, this layer applies important transformations such as removing duplicates, correcting data types, and normalizing text. At the same time, it keeps track of any errors by isolating the rejected rows to maintain data lineage. In short, the Trusted Zone acts as a strong filter, making sure that only clean and reliable data is sent to the next step that is the Exploitation Zone.

2.1 Structured Data Processing

The structured data pipeline focuses on processing the NYC Motor Vehicle Collisions. The system reads the raw data, which was previously stored as a Delta Lake table in MinIO by the Landing Zone. Using PySpark, the script applies rigorous data cleansing before loading the polished data into a highly performant ClickHouse database for analytical querying.

Following the theoretical principles of data cleaning, the key transformations applied are the following:

- **Deduplication (and Data Traceability):** Since the same collision record might appear multiple times across different raw CSV chunks, the script first removes duplicated records based on their primary key (`collision_id`). Furthermore, records missing a valid date are isolated into a separate DataFrame and exported as a timestamped CSV file to a **rejected/** folder in MinIO. This guarantees full data traceability, allowing engineers to audit rejected rows without halting the pipeline.
- **Schema Correction (Type Casting):** To ensure database efficiency, Spark enforces strict mathematical data types on the raw strings. Latitude and longitude are correctly cast to Doubles. Crucially, all casualty metrics (number of injuries and deaths for pedestrians, cyclists, and motorists) are cast to Integers (`UInt16`). If a value is missing or unparseable, the system automatically assigns a safe default value of 0 to prevent downstream analytical calculations from crashing.
- **Standardization (Formatting and Normalization):** The script standardizes both dates and text fields to ensure perfect integration. First, the `crash_date` is parsed from multiple string formats into a standard Spark Date type (`YYYY-MM-DD`), guaranteeing temporal alignment with the enriched semi-structured datasets. Second, geographic text fields (like `borough` and street names) are heavily normalized. The script removes accidental blank spaces and converts all text to uppercase using PySpark's `upper()` and `trim()` functions. This deliberate design choice enables seamless and accurate JOIN operations in the next layers.

Once the PySpark DataFrame is completely clean and standardized, it is cached in memory and inserted into ClickHouse. The script automatically ensures that the necessary database schemas exist, setting up the foundation for two key tables using the MergeTree engine: `trusted_zone.nyc_collisions` and `exploitation_zone.collisions_weather`. By physically ordering the data on the disk by `crash_date` and `collision_id`, ClickHouse provides extremely fast data retrieval.

The first one, `trusted_zone.nyc_collisions`, acts as a clean, standalone baseline dataset. Instead of being discarded after the cleaning phase, it is deliberately preserved in the Trusted Zone to guarantee data lineage and provide visibility into the strictly cleaned collision records. It is important to note that this table does not directly feed the final analytical applications. Its primary purpose is to serve as the structural foundation for the downstream JOIN operations. It is the second table, `exploitation_zone.collisions_weather` (which is pre-staged during this phase), that will ultimately act as the completely enriched source of truth, directly powering both the traffic analysis dashboards (`traffic_collisions_dashboard.py`) and the predictive machine learning models (`risk_prediction_dashboard.py`).

By deliberately persisting the unjoined `trusted_zone` table, the system follows core data engineering best practices: it decouples the cleaning phase from future enrichment phases and guarantees data lineage. At this stage, the structured data is completely refined and ready to be forwarded to the Exploitation Zone, where the actual data integration will take place.

2.2 Semi-Structured Data Processing

Within the Trusted Zone, the system processes the semi-structured JSON files stored in the landing zone. This phase uses Apache Spark to read, clean, normalize, and transform these raw files into structured, flat formats. This process is divided into two main pipelines: one for the Weather Reports and another for the Camera Traffic Metadata. Once the data passes through these pipelines, it is fully standardized and ready for downstream integration in the Exploitation Zone.

(HERE PROBABLY WE CAN DO LIKE, IN THE PREVIOUS EXPLANATION, TO LOOK FOR DEDUPLICATES STORING THEM IN A REJECTED FOLDER IN MinIO)

1. Weather Reports Pipeline

The weather pipeline handles the meteorological data provided by the APIs. The raw JSON files contain deeply nested structures that are difficult to analyze directly. The `semistructured_aggregated_trusted_zone.py` script performs the following key transformations:

- **Flattening and Parsing:** The script extracts deeply nested objects (such as the forecast 'periods') and converts them into a flat, tabular format (rows and columns). If a specific field is missing in the 12-hour forecast (like 'dewpoint'), the system automatically assigns a default value of 0.0 to maintain a consistent schema.

- **Data Cleansing with Regular Expressions:** The raw API provides data with text units (e.g., a wind speed of "15 mph" or a temperature of "72 F"). Spark uses Regular Expressions (`regexp_extract`) to isolate and extract only the numeric values, casting them into mathematical types (Integers and Doubles) so they can be calculated later.
- **Standardization and Enrichment for Data Integration:** The most critical step in this script is preparing the weather data to be joined with the Structured Data (the NYC Motor Vehicle Collisions dataset) in the next zone. To achieve this, temporal data is standardized into a clean YYYY-MM-DD format and explicitly renamed to `crash_date`, matching the exact time variable used in the collisions table. Furthermore, disparate location names retrieved from the API (such as "lower_manhattan" or "harlem") are mapped to official NYC Boroughs using keyword matching (adding them in variable `Borough`). Finally, the system converts these mapped names to uppercase (e.g., "MANHATTAN") so they perfectly match the text format of the geographic variables in the collisions dataset. This meticulous formatting prevents data loss and guarantees seamless JOIN operations in the final Consumption Layer.

2. Camera Traffic Metadata Pipeline

This second pipeline processes the aggregated JSON files generated by the Kafka Consumer, which contain the vehicle detection averages per camera. The `semistructured_aggregated_trusted_zone.py` script applies the following rules:

- **Nested Data Extraction:** The script reads the JSON files and flattens the `avg_per_frame` dictionary. This turns each vehicle category (MotorBike, Pedestrian, Bike, LMV, etc.) into its own independent column. To ensure data integrity, if a camera did not detect a specific type of vehicle (e.g., no e-Rickshaws were found), the system assigns a default value of 0.0 instead of leaving a null value that could cause errors.
- **Borough Assignment and ID Normalization:** Similar to the logic applied in the real-time Consumption Layer, the script standardizes camera identifiers to accurately assign geographic locations. To achieve this, it creates a new `borough` column using PySpark's conditional `when()` function combined with Regular Expressions (`rlike`). Specifically, we extract the exact starting characters of the `camera_id`. Following this rule, IDs starting with "Nd" are explicitly mapped to Manhattan, "NT2" to Brooklyn, "NT4" to Queens, and "SW" to Staten Island. To prevent the pipeline from crashing when encountering a new or malformed camera ID, an `.otherwise("UNKNOWN")` clause is implemented as a safety net. Finally, the system creates a `camera_id_original` column where all letters are converted to uppercase using the `upper()` function, guaranteeing strict string consistency for any future JOIN operations..

Once both JSONs have been fully cleaned, standardized, and validated against their defined schemas, the Spark DataFrames are converted into dictionaries and inserted into

their respective collections (`weather_data` and `camera_aggregates`) within the MongoDB database. At this point, the data is highly reliable and ready to be queried by the Exploitation layer.

2.3 Unstructured Data Processing

The final component of the Trusted Zone is dedicated to processing unstructured data formats, specifically raw text files (`.txt`) and audio recordings (`.wav`) stored in MinIO. Because unstructured data cannot be naturally parsed into standard columns, the system leverages PySpark User-Defined Functions (UDFs) to process the binary content of each file in a distributed manner.

The pipeline guarantees that all unstructured files are standardized, cleaned, and validated before being saved into the Trusted MinIO buckets. It is divided into two distinct processing streams:

1. Text Processing Pipeline

The text cleaning script applies information-preserving transformations to prepare the text for potential Natural Language Processing (NLP) or text mining tasks. The applied rules include:

- **Encoding Validation:** The system attempts to read the file using UTF-8 encoding (falling back to Latin-1 if necessary). If the file is completely unreadable or empty, it is flagged as corrupted.
- **Whitespace and Case Normalization:** To ensure consistency, all text is converted to lowercase. The script removes trailing/leading whitespaces, replaces tabs and multiple spaces with a single space, and collapses multiple consecutive blank lines into one.

2. Audio Processing Pipeline

Raw audio files often come with disparate configurations (different bit rates, channels, etc.) that can crash Machine Learning models. The audio pipeline enforces a strict standardization process:

- **File Validation:** The script reads the RIFF header of the `.wav` file to detect corruption. It also checks the duration of the audio, automatically rejecting clips that are shorter than a predefined minimum threshold (`TRUSTED_AUDIO_MIN_DURATION_SECONDS`).
- **Signal Standardization:** Using Python's `audioop` and `wave` libraries, all valid files are standardized to a strict baseline. If necessary, the audio is converted to 16-bit PCM, mixed down from stereo to mono (1 channel), and resampled to the project's target sample rate. This ensures that all audio files share the exact same acoustic properties.

Following the same data engineering best practices established in the structured pipeline, this unstructured pipeline strictly enforces data lineage. Files that fail validation (e.g., corrupted WAVs, unreadable text, or excessively short audio clips) are not deleted. Instead, they are isolated and saved into a dedicated **skipped/** directory within MinIO, tagged with a **skip_reason** for future auditing. Successfully cleaned and standardized files are securely persisted back into MinIO, residing in the Trusted Zone ready for downstream consumption.

3 Exploitation Zone

Once the datasets have been rigorously cleaned and standardized in the Trusted Zone, they are moved to the Exploitation Zone. According to the architectural requirements, the primary goal of this layer is curation: organizing, modelling, and persisting the data in a way that minimizes query complexity and latency for the final analysts. Following data engineering best practices, this layer acts as a staging ground to produce wide, query-friendly tables that eliminate the need for expensive operations in the downstream Consumption Layer.

3.1 Structured and Semi-Structured Integration

This pipeline is responsible for generating the core analytical table of the project. To achieve the maximum analytical value, the architecture relies on two fundamental operations defined by the zone's theoretical constraints: structural denormalization (Organizing) and the calculation of new analytical assets (Computing new data).

Organizing: Denormalization and Cross-Database JOINS

Analytical workloads benefit significantly from pre-joined structures. Instead of keeping the data isolated, PySpark acts as a distributed processing engine to integrate data from multiple database technologies into a single, wide schema:

- **Base Structured Data:** It reads the cleaned NYC collision records directly from the ClickHouse Trusted Zone.
- **Semi-Structured Enrichment:** It retrieves the processed meteorological data and the traffic camera metadata, which are stored as document collections in MongoDB.

Once loaded into Spark DataFrames, the system performs a distributed `LEFT JOIN` to denormalize the model. The data is aligned using the standardized geographic key (`borough`). By doing this, every individual motor vehicle collision row is directly appended with the environmental and traffic conditions corresponding to that exact location. This specific design choice guarantees that the data is served as a flat, highly cohesive unit, minimizing the need for complex joins at consumption time.

Computing New Data: KPIs and Aggregations

Rather than simply joining raw metrics, this pipeline enhances the analytical value of the datasets by computing and storing key performance indicators (KPIs) and summary statistics. By maintaining a record of these aggregates in the Exploitation Zone, analysts gain immediate access to insights without needing to recompute them in every query.

The `aggregate_by_borough` functions perform the following calculations before the JOIN operations:

- **Meteorological Averages:** The script groups the MongoDB weather data by borough and computes the spatial average (`avg`) for critical variables, including temperature, wind speed, precipitation probability, and humidity.
- **Traffic Volume KPIs:** For the camera data, the script computes the average detection count for each specific vehicle category (e.g., MotorBikes, Pedestrians, LMVs). Furthermore, it dynamically computes a brand-new, higher-level metric—`cam_avg_total_traffic`—by mathematically summing all individual vehicle categories into a single KPI prior to aggregation.

After the denormalization and KPI generation processes are complete, the resulting enriched DataFrame is inserted back into ClickHouse. The script utilizes `ALTER TABLE` commands to dynamically ensure the schema is prepared for the new columns. By persisting this pre-aggregated dataset using ClickHouse’s `MergeTree` engine, the data is completely curated, optimized, and ready to be seamlessly accessed by the tools in the Consumption Layer.

3.2 Semi-Structured Data Enrichment

Beyond the massive denormalized table used for strict vehicle collision analysis, the architecture includes a secondary pipeline specifically dedicated to enriching the semi-structured camera traffic data. Rather than forcing all datasets into a single relational table, this pipeline leverages MongoDB as an agile NoSQL exploitation store.

This decision guarantees that the traffic metadata retains its flexible schema while still being structurally optimized for independent analytical queries in the Consumption Layer.

According to the theoretical design of the Exploitation Zone, this layer must serve as a staging ground for calculating higher-level metrics that reflect strategic insights. The script retrieves the trusted camera documents from MongoDB and computes a brand-new asset prior to consumption: `total_traffic`.

Once the traffic KPI is generated, the enriched dataset is persisted back into MongoDB under a new namespace (`exploitation_zone.camera_aggregates`). Furthermore, this script on every execution it drops the existing collection and safely rebuilds it from the Trusted Zone source. This ensures absolute structural consistency and prevents duplicate aggregations, leaving the documents fully prepared for decoupled analytical visualization.

3.3 Unstructured Data Extraction

Unlike structured records, raw text and audio cannot be natively queried with SQL. To comply with the architectural requirement of "Computing new data," this pipeline extracts structured, semantic representations from the raw files, ensuring that complex analytical algorithms do not need to be repeatedly evaluated during the downstream consumption phase.

In a standard Machine Learning environment, generating vector embeddings typically involves heavy deep learning models (such as Transformers via PyTorch), which introduce significant computational overhead, GPU dependencies, and model-download latency. For this specific data engineering pipeline, the architecture prioritizes speed, efficiency, and deterministic execution over deep semantic richness.

Instead of heavy neural networks (the docker was heavy, the system implements dependency-light vector representations using pure NumPy:

- **Text Encoder:** A hashing-trick vectorizer is applied over unigrams and bigrams to generate a 128-dimensional float vector. Each token is mapped via a stable, process-independent 64-bit hash.
- **Audio Encoder:** The mono PCM audio signal is split into 16 equal windows. For each window, core acoustic statistics are calculated (Root Mean Square energy, mean amplitude, standard deviation, and zero-crossing rate), resulting in a precise 64-dimensional temporal envelope.

Both methods produce L2-normalized float vectors that behave exactly like transformer embeddings when queried via cosine similarity.

Organizing and Dual Persistence: Milvus and MinIO

Because unstructured data requires both vector similarity search capabilities and physical file retrieval, the pipeline distributes the extraction workload using PySpark User-Defined Functions (UDFs) and implements a dual-sink storage strategy:

- **Vector Database (Milvus):** The generated mathematical embeddings, along with their explicitly parsed metadata (e.g., text categories, audio durations, sample rates), are directly inserted into Milvus collections (`text_embeddings` and `audio_embeddings`). This NoSQL database allows downstream ML tools to retrieve similar assets in milliseconds.
- **Object Storage (MinIO):** Simultaneously, the physical `.wav` and `.txt` files are copied into the Exploitation Zone bucket in MinIO. This guarantees that the original raw assets remain highly available for the consumption dashboards (e.g., Streamlit).

By executing this pipeline, the Exploitation Zone successfully transforms opaque files into highly queried, ML-ready assets without the overhead of heavy deep learning frameworks, completing the data curation phase and leaving all sources completely ready for the ultimate Consumption Layer.

4 Data Consumption

After the ingestion, cleaning, and organization phases, our architecture reaches its final stage: the Consumption Layer. As defined in the project requirements, the main goal here is not to build perfect analytical models, but to prove that our data pipeline works from start to finish and can successfully feed external applications.

This layer takes the prepared data from the Exploitation Zone (tables, NoSQL collections, and embeddings) and puts it to practical use. To meet the highest standards, we avoided using static placeholders. Instead, we implemented fully functional mechanisms. By connecting directly to our databases, this layer powers real interactive dashboards and machine learning workflows, showing that our engineered data is completely ready for final analysis.

4.1 Predictive Analysis: Collision Risk Modeling

To leverage the structured data curated in the Exploitation Zone, we implemented a predictive analysis system that acts as our primary Machine Learning consumption mechanism. This module connects directly to the `ClickHouse` database to read the wide `collisions_weather` table, enabling the training of real-time supervised learning models.

Machine Learning Implementation

The core objective of this module is to predict whether a traffic collision will result in casualties (injuries or fatalities). This is modeled as a binary classification task, where we employ two robust algorithms: **Random Forest** and **XGBoost**.

The pipeline performs the following steps:

- **Feature Engineering:** The system processes temporal variables (e.g., hour of the day, day of the week) and spatial data (label-encoded boroughs), combined with environmental factors like precipitation and wind speed.
- **Model Training:** Using *Scikit-Learn*, we train both models on the curated collision dataset. The system calculates performance metrics such as *AUC-ROC*, *Precision*, and *Recall* to ensure the model's reliability in identifying high-risk areas.
- **Feature Importance Analysis:** To provide transparency, the dashboard visualizes which factors (e.g., traffic volume vs. precipitation) contribute most significantly to the risk prediction.

Interactive Risk Simulator

To make this analytical asset accessible to end-users, we integrated an Interactive Risk Simulator within our Streamlit interface. This simulator acts as the consumption point

for the trained models: it allows users to adjust real-world conditions—such as increasing traffic volume or changing weather probability—via interactive sliders.

By feeding these modified parameters directly into the trained **XGBoost** and **Random Forest** models, the simulator provides an immediate prediction of how collision risk shifts across different boroughs. This demonstrates that our data pipeline is not just a storage system, but a dynamic, production-ready environment that can effectively power predictive analytics and support decision-making processes.

4.2 Analytical Dashboard: Traffic Collision Insights

To provide a high-level overview of traffic patterns, we implemented an analytical dashboard using **Streamlit** and **Plotly**. While the predictive module focuses on Machine Learning, this dashboard is designed to provide actionable business intelligence by transforming the massive datasets stored in **ClickHouse** into intuitive visual trends.

Dashboard Functionality

The dashboard directly queries the `exploitation_zone.collisions_weather` table to power two primary analytical views:

- **Monthly Trend Analysis:** This module visualizes collision frequency over time. It features a multi-select interactive filter that allows analysts to compare collision trends between different New York boroughs (e.g., Manhattan vs. Brooklyn) over specific periods. By using a line chart with time-series aggregation (`toStartOfMonth`), the dashboard helps identify seasonal patterns or spikes in collision activity.
- **Contributing Factor Breakdown:** This module identifies the root causes of traffic accidents. Through an interactive select box, users can filter by borough to display the top 10 contributing factors (e.g., "Driver Inattention," "Failure to Yield"). The dashboard dynamically recalculates these rankings based on the selected borough, providing local insights into traffic safety challenges.

Value for Decision Making

By connecting this dashboard to the Exploitation Zone, we ensure that stakeholders have immediate access to the most up-to-date information without the need for complex manual queries. The integration of `st.cache_data` ensures that the application remains responsive even when handling historical datasets, proving that our data management architecture is capable of supporting professional-grade, interactive business intelligence tools.

4.3 NLP and STT

To do

4.4 Real-Time Analysis and Alerts

As explained in the previous deliverable, our system uses a Kafka Producer that simulates sending data from different traffic cameras in New York. This data is then ingested by the consumer (`unstructured_data_image_consumer.py`), which processes the real-time metadata of the images and sends it as aggregated JSONs to the Trusted Zone. On the other hand, to make use of the raw images immediately and fulfill our real-time analytics objective, we created an additional script in this layer called `spark_streaming_cv_alerts.py` that instead of putting it in the consumption layer folder is put it in the.

This program uses **Apache Spark Streaming** to analyze images in real time, detect vehicles using Artificial Intelligence (AI), and generate **automatic traffic alerts**. The process is divided into the following five steps:

1. **Image Reception and Preparation:** The system connects to Kafka to collect image messages. To ensure temporal accuracy, we configure the Spark session to the **America/New_York** time zone, ensuring all event timestamps reflect local NYC time. The system then extracts the exact timestamp and the image data (in Base64 format). Finally, using the OpenCV library, the program decodes the Base64 string and reconstructs the original image for subsequent analysis.
2. **Vehicle Detection with AI:** To find vehicles in the photos, we use a computer vision model called MobileNet-SSD. This model analyzes the image and looks for specific objects: bicycles, buses, cars, and motorbikes. After trying very high confidence values, as this model only accepts 300x300 images, it is not able to detect vehicles properly. That is why we added a lower confidence level of 0.1 to improve detection rates.
3. **Data Enrichment and Simulation:** After determining the vehicle count, the program identifies the New York district (**Borough**) associated with the camera, following the same logic used in the Trusted Zone for semi-structured . For example, if the ID starts with "Nd", the system assigns the camera to Manhattan; if it starts with "NT2", it is assigned to Brooklyn, and so on. Any unrecognized camera ID is labeled as "UNKNOWN" to prevent errors. Finally, all camera IDs are converted to uppercase to maintain data consistency, and the system assigns a representative street (like Broadway or 5th Avenue). Additionally, to effectively demonstrate the alert system under heavy traffic conditions, the code adds a baseline of 19 vehicles to the actual detected count. This simulation step is necessary because when applying the model to the available images the model is not able to detect more than 2 cars, that is why we add 19
4. **Time Grouping (15-Second Windows):** Spark Streaming organizes the information into groups or "time windows" of 15 seconds. It is very important to note that this grouping is done using the real time when the photo was taken (Event Time), and not the time it takes the computer to process it. The system groups the data by district and by camera to find the maximum number of vehicles detected in those 15 seconds. Crucially, the system links this maximum vehicle count to its corresponding image data. This guarantees that the final alert contains the exact photograph that registered the highest number of vehicles within that window.

5. **Alert Generation and Storage Optimization:** Finally, the program filters the data and only keeps the records where there are 20 or more vehicles. To avoid running the heavy AI calculations twice when outputting to both the screen and a file, we temporarily save the results in RAM using the `.persist()` function. This allows the system to efficiently print the alert in the console and save the data to disk. For every critical alert, the system automatically creates an organized, nested folder structure (`Alerts/Borough/Timestamp/`). Inside this directory, it saves both a text report (`.txt`) with actionable information for traffic authorities, and the decoded image (`.jpg`) showing the exact traffic jam. This provides immediate visual proof of the congestion. Afterwards, the memory is automatically cleared using `.unpersist()` for the next group of photos.

5 Data Governance Strategy

TO DO Some data governance strategies we have thought (but they are still not implemented) are:

- Automated validation rules within the pipeline (e.g., rejecting temperature records outside the -40°C to 50°C range). Invalid data is diverted to a `skipped/` folder to avoid ingestion errors.
- Clear definition of roles: Data Engineers manage the full pipeline, while Data Scientists only have read-only access to the Exploitation Zone to protect the "source of truth."
- Every dataset in the Exploitation Zone includes a `README.json` file (containing owner, schema, and last update date), ensuring all team members can identify the data independently, making it self-documented.
- Backup Policy: We perform weekly exports of the `ClickHouse` tables to SQL format. These backups are stored in a secure, separate location, ensuring that in the event of a system failure, we can restore the full analytical state of the project.